UNIVERSIDAD TECNOLÓGICA DE XICOTEPEC DE JUÁREZ

Ingeniería en Tecnologías de la Información e Innovación Digital

Optativa I: Bases de Datos en la Nube

Reporte técnico individual

Examen práctico · Unidad 1

Implementación y administración de bases de datos en la nube

Estudiante	Adrian Rios Gomez
Matrícula	240165
Grupo / equipo	7A / Nexus Software
Docente	M.T.I. Marco A. Ramírez Hernández
Proveedor del VPS	Hostinger
IP del VPS	179.236.78.124
Puerto PostgreSQL / MySQL	40165 / 40166 (pgAdmin Web: 40167)
Repositorio / rama	Manual_VPS_BDs_Relacionales_240165 (privado) · Manual_VPS_BDs_Relacionales
Fecha	23 de octubre de 2026

Índice

1. Descripción de la práctica..... 3

2. Desarrollo..... 4

2.1 Adquisición y configuración del VPS..... 4
2.2 Cuentas del sistema operativo y permisos..... 5
2.3 Instalación de herramientas..... 5
2.4 Despliegue con Docker Compose..... 6
2.5 Puertos asignados y conexiones remotas..... 10
2.6 Roles, cuentas y privilegios..... 13
2.7 Base de datos y objetos relacionales..... 20

3. Pruebas y resultados..... 27

Caso 01: el superadmin inserta 3 usuarios..... 28
Caso 02: el usuario sin rol inserta 3 usuarios..... 28
Caso 03: support inserta 3 usuarios..... 29
Caso 04: el superadmin modifica y elimina un usuario ajeno..... 30
Caso 05: el usuario sin rol intenta modificar y eliminar..... 30
Caso 06: support modifica y elimina..... 31
Caso 07: el superadmin inserta 2 productos..... 31
Caso 08: el vendedor 1 inserta 5 productos..... 32
Caso 09: el vendedor 2 importa el CSV de 5000 productos..... 32
Resumen y trazabilidad tras los casos..... 34
Primer intento fallido del caso 09..... 34

4. Control de versiones..... 35

5. Conclusiones personales..... 41

1. Descripción de la práctica

Contexto. En la Unidad 1 de la asignatura Optativa I: Bases de Datos en la Nube se despliega una infraestructura de bases de datos relacionales sobre un servidor privado virtual (VPS) compartido por un equipo de seis integrantes. Cada estudiante trabaja con un entorno propio e independiente dentro del mismo servidor, con sus propios contenedores, puertos y volúmenes.

Objetivo. Desplegar, configurar, administrar y verificar PostgreSQL, MySQL y pgAdmin Web con Docker Compose; publicar los servicios en los puertos derivados de la matrícula; administrar los roles, las cuentas y los privilegios del contexto de comercio electrónico; crear la base de datos `db_test` con sus tablas, triggers y vistas de trazabilidad; ejecutar los casos de prueba sobre `tb_users` y `tb_products`; y entregar el trabajo con control de versiones en un repositorio privado.

Tecnologías utilizadas.

- Ubuntu 26.04 LTS como sistema operativo del VPS, proporcionado por Hostinger.
- Docker 29.1 y Docker Compose 5.6 para el despliegue de los servicios.
- PostgreSQL 17.11, MySQL 8.4.13 y pgAdmin 4 Web como motores y herramienta de administración.
- Git 2.53 y GitHub para el control de versiones.
- Bash, SQL (PL/pgSQL y SQL de MySQL) y Python para los scripts, las pruebas y el cliente de conexión externo.

Alcance. El ejercicio incluye los dos motores con los mismos roles, cuentas, esquema de 16 tablas, triggers y vistas, las pruebas funcionales con la cuenta real de cada rol y la demostración de conexión remota desde un cliente externo. La cuenta del estudiante en el VPS no cuenta con `sudo`, por lo que el despliegue se realizó con Docker, ya que la cuenta pertenece al grupo `docker`.

Evidencias. Las imágenes de terminal de este reporte reproducen, con el mismo contenido, la salida real que se obtuvo al ejecutar cada comando en el VPS o en la computadora del estudiante, y los comandos y el código aparecen siempre como imagen. Las capturas de pgAdmin Web y de GitHub son capturas de pantalla directas del navegador.

2. Desarrollo

2.1 Adquisición y configuración del VPS

El VPS fue adquirido con el proveedor Hostinger y se encuentra habilitado y accesible desde Internet por SSH (puerto 22) y por los puertos publicados de los servicios. La verificación general del sistema operativo, la dirección IP y los recursos disponibles se realizó con el script `scripts/08_info_vps.sh`. El servidor cuenta con 2 núcleos de CPU, 7.7 GiB de memoria RAM y 96 GB de disco, de los cuales solo se usa el 7 %.

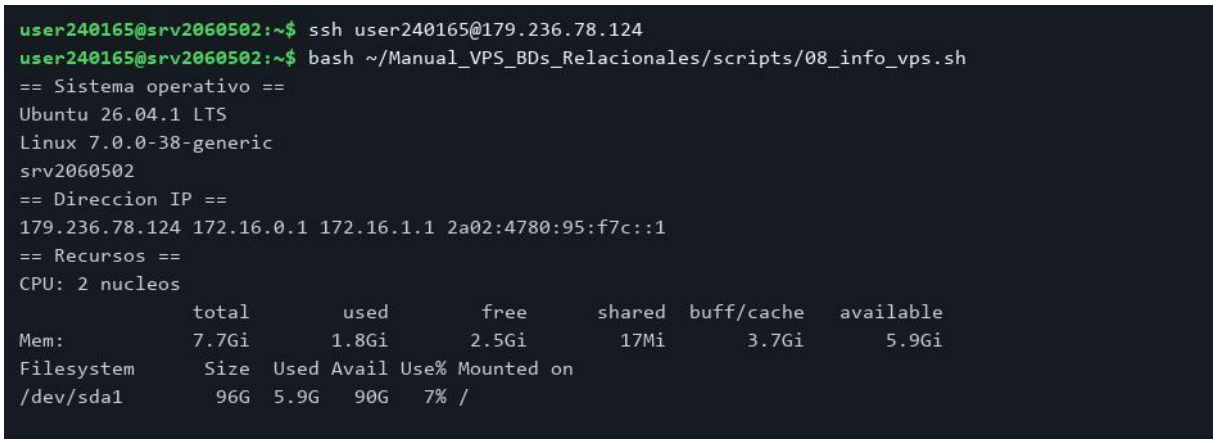


Figura 1. Sistema operativo, dirección IP y recursos del VPS.

2.2 Cuentas del sistema operativo y permisos

Se crearon seis cuentas personales de acceso al sistema operativo, una por cada integrante del equipo. Todas pertenecen al grupo `users` y al grupo `docker`, que les permite administrar sus propios contenedores sin privilegios de superusuario; ninguna cuenta tiene `sudo`. Cada estudiante trabaja en su directorio personal y publica sus servicios en puertos distintos derivados de su matrícula, de modo que las responsabilidades quedan separadas.

```
user240165@srv2060502:~$ getent passwd | awk -F: '$3>=1000 && $3<65000 {print $1}' | while read u;
                         do echo "$u: $(id -nG $u)"; done
user240628: user240628,users,docker
user240808: user240808,users,docker
user240763: user240763,users,docker
user240161: user240161,users,docker
user240165: user240165,users,docker
user240201: user240201,users,docker
user240165@srv2060502:~$ id
uid=1004(user240165) gid=1004(user240165) groups=1004(user240165),100(users),986(docker)
```

Figura 2. Cuentas del sistema operativo del equipo y sus grupos.

2.3 Instalación de herramientas

Git, Docker Engine y Docker Compose están instalados y funcionan correctamente en el VPS. Con Docker Compose se despliega todo el proyecto y con Git se administra el control de versiones del trabajo.

```
user240165@srv2060502:~$ git --version
git version 2.53.0
user240165@srv2060502:~$ docker --version
Docker version 29.1.3, build 29.1.3-0ubuntu4.1
user240165@srv2060502:~$ docker compose version
Docker Compose version v5.6.0
```

Figura 3. Versiones de Git, Docker y Docker Compose instaladas en el VPS.

2.4 Despliegue con Docker Compose

El proyecto `docker-compose.yml` es propio e independiente del de los demás compañeros: el proyecto se llama `bdn240165` y define sus propios contenedores, red y volúmenes. Incluye PostgreSQL 17, MySQL 8.4 y pgAdmin Web. Las contraseñas no se escriben en el archivo: se leen de un archivo `.env` que no se sube al repositorio.

```yaml
name: bdn240165

services:
  postgres:
    image: postgres:17
    container_name: postgres40165
    restart: unless-stopped
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    command: ["postgres", "-c", "port=40165", "-c", "listen_addresses=*"]
    ports:
      - "40165:40165"
    volumes:
      - ${DATA_DIR}/postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -p 40165 -U postgres"]
      interval: 10s
      timeout: 5s
      retries: 10

  mysql:
    image: mysql:8.4
    container_name: mysql40166
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_ROOT_HOST: "%"
    command: ["--port=40166", "--local-infile=1", "--mysql-native-password=ON"]
    ports:
      - "40166:40166"
    volumes:
      - ${DATA_DIR}/mysql_data:/var/lib/mysql
    healthcheck:
      test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -P 40166 -uroot -p$$MYSQL_ROOT_PASSWORD
```

Figura 4. Archivo docker-compose.yml con los tres servicios. (continuación 1 de 2)

```
          --silent"]
      interval: 10s
      timeout: 5s
      retries: 10

  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: pgadmin240165
    restart: unless-stopped
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
    ports:
      - "40167:80"
    volumes:
      - pgadmin_data:/var/lib/pgadmin
    depends_on:
      postgres:
        condition: service_healthy

volumes:
  pgadmin_data:
```

Figura 5. Archivo docker-compose.yml con los tres servicios. (continuación 2 de 2)

El despliegue se ejecuta con `docker compose up -d`. Los datos de PostgreSQL y MySQL persisten en los directorios `~/postgres_data` y `~/mysql_data`, y los de pgAdmin en el volumen `bdn240165_pgadmin_data`, por lo que no se pierden al recrear los contenedores.

```
user240165@srv2060502:~$ cd ~/Manual_VPS_BDs_Relacionales
user240165@srv2060502:~$ docker compose up -d
Recreando contenedores con el .env nuevo (los datos persisten en volumenes)...
 Container postgres40165 Started
 Container postgres40165 Waiting
 Container mysql40166 Started
 Container postgres40165 Healthy
mysql40166: healthy | postgres40165: healthy
```

Figura 6. Creación de los contenedores con docker compose up -d.

```
user240165@srv2060502:~$ docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}'
NAMES           IMAGE                  STATUS                    PORTS
postgres40165   postgres:17            Up 14 seconds (healthy)   5432/tcp,
    0.0.0.0:40165->40165/tcp, [::]:40165->40165/tcp
mysql40166      mysql:8.4              Up 14 seconds (healthy)   3306/tcp, 33060/tcp,
    0.0.0.0:40166->40166/tcp, [::]:40166->40166/tcp
pgadmin240165   dpage/pgadmin4:latest  Up 17 minutes             443/tcp, 0.0.0.0:40167->80/tcp,
    [::]:40167->80/tcp
user240165@srv2060502:~$ docker volume ls | grep bdn240165
local     bdn240165_pgadmin_data
user240165@srv2060502:~$ ls -ld ~/mysql_data ~/postgres_data
drwxr-xr-x  8 dnsmasq root 4096 Oct 23 21:02 /home/user240165/mysql_data
drwx------ 19 dnsmasq root 4096 Oct 23 21:02 /home/user240165/postgres_data
```

Figura 7. Contenedores en ejecución, volúmenes y datos persistentes.

```
user240165@srv2060502:~$ for c in postgres40165 mysql40166 pgadmin240165; do docker logs --since
                         24h $c 2>&1 | grep -ci 'fatal\|panic\|critical'; done
-- postgres40165: 0 lineas con FATAL/PANIC/CRITICAL
-- mysql40166: 0 lineas con FATAL/PANIC/CRITICAL
-- pgadmin240165: 0 lineas con FATAL/PANIC/CRITICAL
```

Figura 8. Revisión de los registros: ningún error crítico en los tres contenedores.

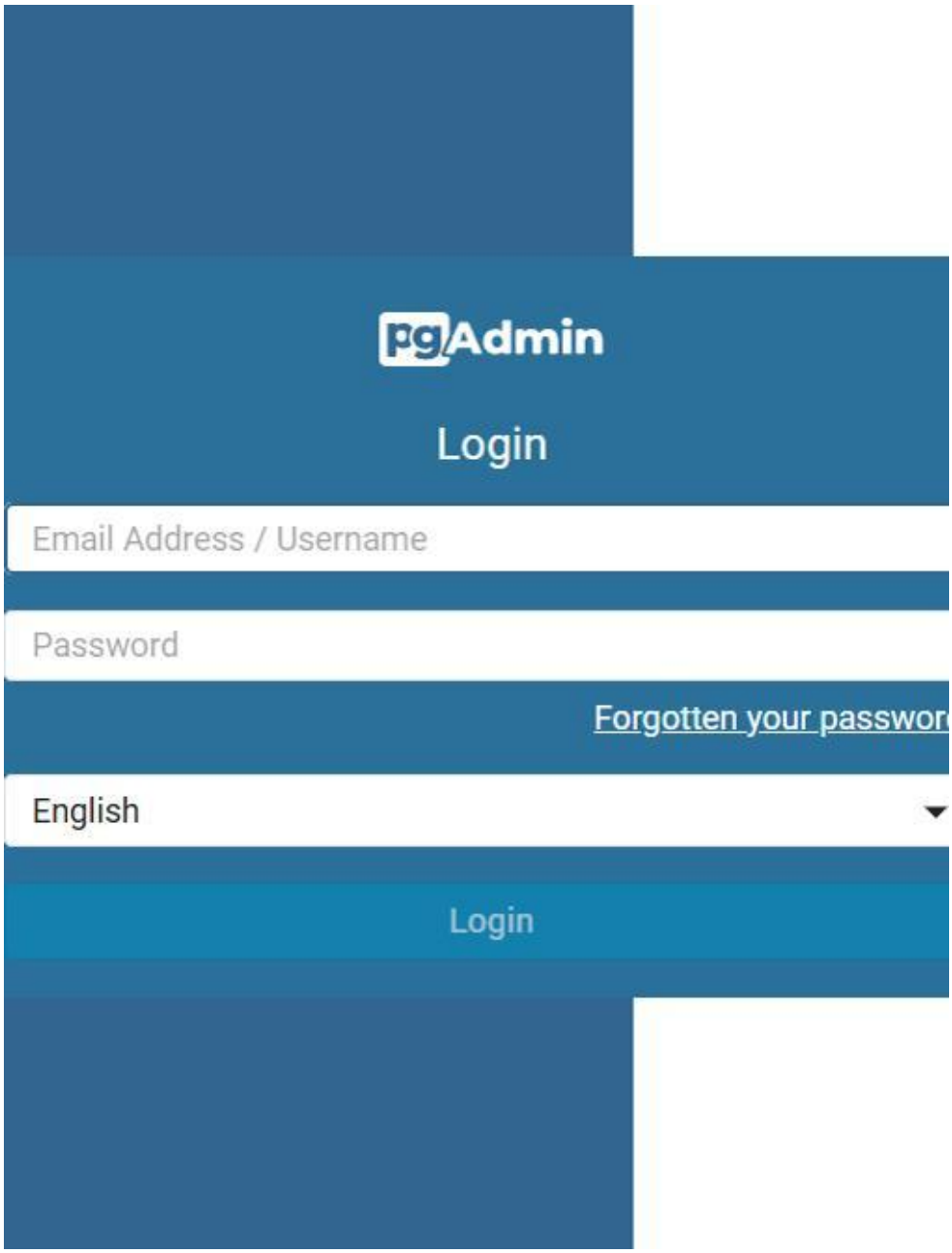


Figura 9. pgAdmin Web publicado en el puerto 40167 (http://IP_DEL_VPS:40167/login).

2.5 Puertos asignados y conexiones remotas

La matrícula 240165 genera el puerto **40165** para PostgreSQL (los cinco dígitos de la matrícula) y el puerto **40166** para MySQL (la matrícula más uno). Ambos están dentro del intervalo válido de puertos TCP (1 a 65535). pgAdmin Web se publica en el 40167. Los puertos por defecto 3306 y 5432 no están publicados, por lo que no se generan conflictos con los servicios de los demás integrantes.

Servicio	Puerto publicado	Puerto por defecto	Observación
PostgreSQL 17	40165	5432 (cerrado)	Cinco dígitos de la matrícula
MySQL 8.4	40166	3306 (cerrado)	Matrícula más uno
pgAdmin 4 Web	40167	80 (interno)	Interfaz web de administración

Ambos motores están configurados para aceptar conexiones remotas autorizadas: PostgreSQL con `listen_addresses=*` y autenticación `scram-sha-256`, y MySQL con cuentas definidas para cualquier origen (`'usuario'@'%'`). La conexión de prueba se realizó desde un cliente externo, la computadora del estudiante, sin usar SSH ni PuTTY para ejecutar las consultas, y con la cuenta administradora de cada motor. Una contraseña incorrecta es rechazada por ambos servidores.

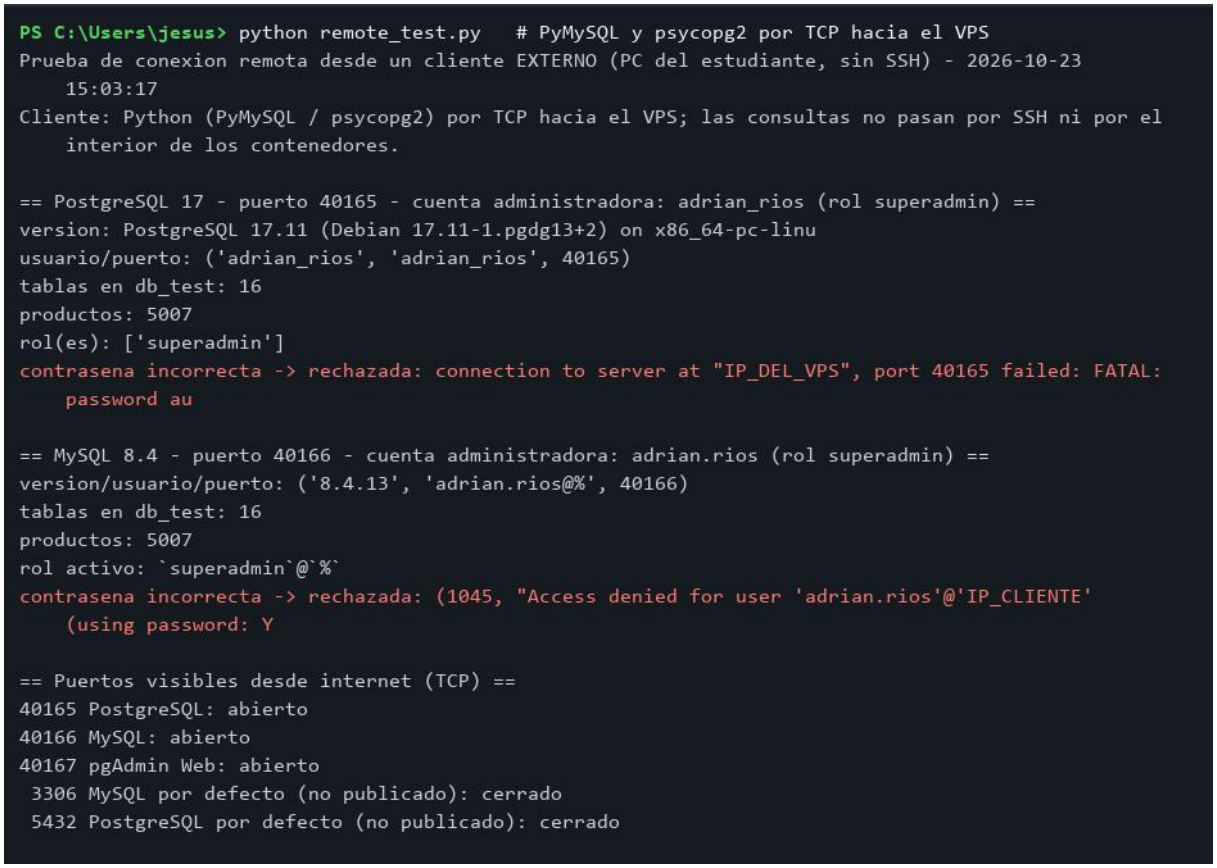


Figura 10. Conexión remota a PostgreSQL y MySQL desde un cliente externo y puertos visibles desde Internet.

```
PS C:\Users\jesus> & 'mysql.exe' -h IP_DEL_VPS -P 40166 -u adrian.rios db_test -t -e "SELECT VERSION() AS version, CURRENT
    _USER() AS usuario, @@port AS puerto, CURRENT_ROLE() AS rol_activo; SHOW FULL TABLES; SELECT COUNT(*) AS productos FRO
    M tb_products;"
+---------+---------------+--------+------------------+
| version | usuario       | puerto | rol_activo       |
+---------+---------------+--------+------------------+
| 8.4.13  | adrian.rios@% |  40166 | `superadmin`@`%` |
+---------+---------------+--------+------------------+
+-----------------------------------------+------------+
| Tables_in_db_test                       | Table_type |
+-----------------------------------------+------------+
| tb_carts                                | BASE TABLE |
| tb_logs                                 | BASE TABLE |
| tb_people                               | BASE TABLE |
| tb_products                             | BASE TABLE |
| tb_seller_orders                        | BASE TABLE |
| tb_user_addresses                       | BASE TABLE |
| tb_users                                | BASE TABLE |
| tbc_categories                          | BASE TABLE |
| tbc_payment_methods_types               | BASE TABLE |
| tbd_address_components                  | BASE TABLE |
| tbd_cart_items                          | BASE TABLE |
| tbd_order_items                         | BASE TABLE |
| tbd_products_categories                 | BASE TABLE |
| tbd_products_users_providers            | BASE TABLE |
| tbd_user_payment_methods                | BASE TABLE |
| tbd_users_wishlist_products             | BASE TABLE |
| vw_total_products_by_category           | VIEW       |
| vw_total_products_by_category1          | VIEW       |
| vw_trazabilidad_address_components      | VIEW       |
| vw_trazabilidad_cart_items              | VIEW       |
| vw_trazabilidad_carts                   | VIEW       |
| vw_trazabilidad_categorias              | VIEW       |
| vw_trazabilidad_logs                    | VIEW       |
| vw_trazabilidad_order_items             | VIEW       |
| vw_trazabilidad_payment_methods_types   | VIEW       |
| vw_trazabilidad_people                  | VIEW       |
| vw_trazabilidad_productos               | VIEW       |
| vw_trazabilidad_products_categories     | VIEW       |
| vw_trazabilidad_products_users_providers | VIEW       |
| vw_trazabilidad_seller_orders           | VIEW       |
| vw_trazabilidad_user_addresses          | VIEW       |
| vw_trazabilidad_user_payment_methods    | VIEW       |
| vw_trazabilidad_users_wishlist_products | VIEW       |
| vw_trazabilidad_usuarios                | VIEW       |
+-----------------------------------------+------------+
+-----------+
| productos |
+-----------+
|      5007 |
+-----------+
PS C:\Users\jesus>
```

Figura 11. Conexión remota a MySQL 8.4 desde la PC del estudiante con mysql.exe y la cuenta adrian.rios (rol superadmin).

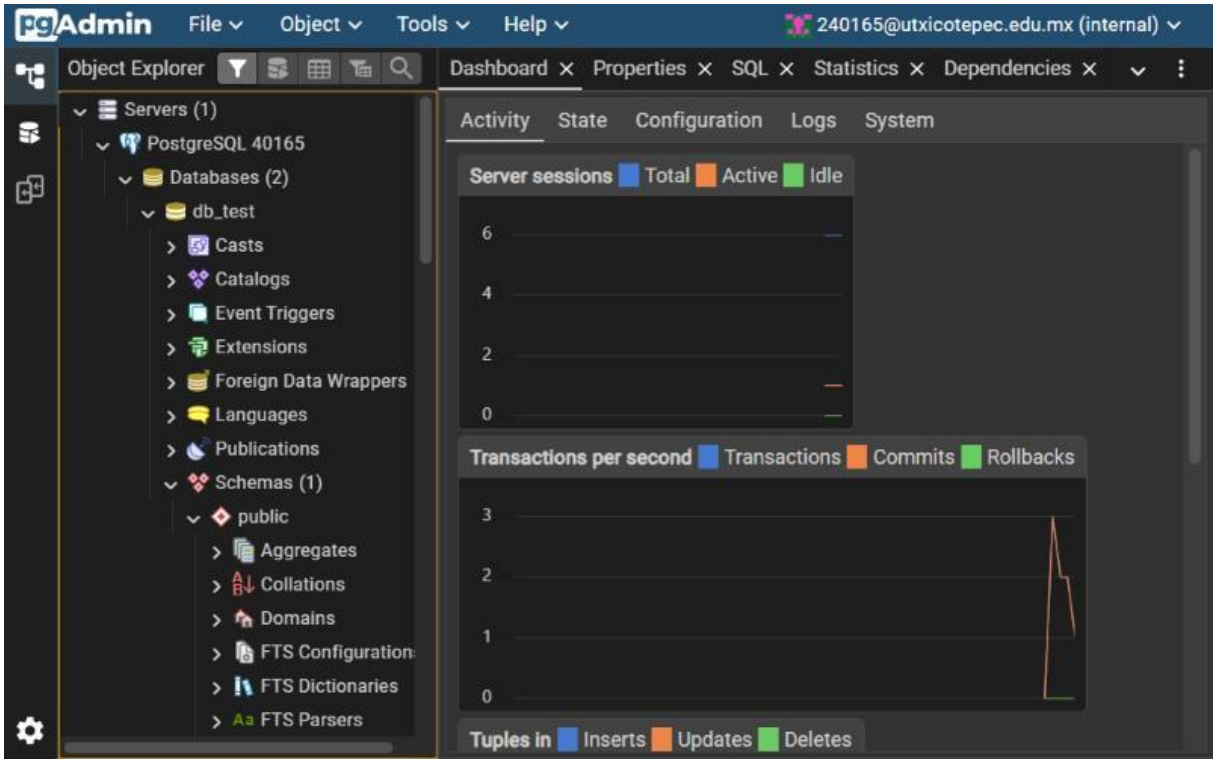


Figura 12. pgAdmin Web (accedido por Internet) conectado a PostgreSQL 17 con la cuenta adrian_rios: servidor PostgreSQL 40165, base db_test, esquema public.

Firewall y restricciones de acceso. La cuenta del estudiante no tiene `sudo`, por lo que no es posible listar las reglas de `ufw`. Las restricciones efectivas son las siguientes: solo se publican los puertos 40165, 40166 y 40167; los puertos por defecto están cerrados desde Internet; todas las cuentas exigen contraseña; y los privilegios están limitados por rol, como se muestra en la sección siguiente.

2.6 Roles, cuentas y privilegios

Los siete roles del contexto de comercio electrónico (`superadmin`, `admin`, `seller`, `buyer`, `support`, `common` y `not_registered`) se crearon tanto en PostgreSQL (`sql/postgresql.sql`) como en MySQL (`sql/mysql.sql`), con las mismas reglas de negocio de la Práctica 02.

Rol	Privilegios
superadmin	Todos los privilegios, incluida la administración de roles y de bases de datos.
admin	Todos los privilegios sobre la base de datos db_test.
support	SELECT y UPDATE en tb_users y tb_products; SELECT, INSERT y UPDATE en tbc_categories; ejecución de rutinas (registro y baja lógica). No puede eliminar registros.

Rol	Privilegios
seller	SELECT, INSERT y UPDATE en tb_products.
buyer y common	SELECT en tb_products y tbc_categories (consulta del catálogo).
not_registered	INSERT en tb_users y tb_people (solo puede registrarse).

A continuación se muestra el código SQL que crea los roles, las cuentas personales y asigna los privilegios en PostgreSQL y en MySQL.

```
/* ============================================================
   1. ROLES (grupos sin login)
   ============================================================ */
DO $$
DECLARE r text;
BEGIN
  FOREACH r IN ARRAY
      ARRAY['superadmin','admin','seller','buyer','support','common','not_registered']
  LOOP
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = r) THEN
      EXECUTE format('CREATE ROLE %I NOLOGIN', r);
    END IF;
  END LOOP;
END $$;
ALTER ROLE superadmin CREATEROLE CREATEDB;

/* ============================================================
   2. CUENTAS PERSONALES Y DE APLICACION
   ============================================================ */
DO $$
DECLARE u text;
BEGIN
  FOREACH u IN ARRAY ARRAY['adrian_rios','marco_ramirez','carlos_rene','juan_pablo',
                           'erick_uriel','jose_maria','app_web','app_movil']
  LOOP
    IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = u) THEN
      EXECUTE format('CREATE ROLE %I LOGIN', u);
    END IF;
  END LOOP;
END $$;

ALTER ROLE adrian_rios   PASSWORD :'pw_personal';
ALTER ROLE marco_ramirez PASSWORD :'pw_personal';
ALTER ROLE carlos_rene   PASSWORD :'pw_personal';
```

Figura 13. Código SQL de PostgreSQL: creación de roles y cuentas personales. (continuación 1 de 2)

```
ALTER ROLE juan_pablo    PASSWORD :'pw_personal';
ALTER ROLE erick_uriel   PASSWORD :'pw_personal';
ALTER ROLE jose_maria    PASSWORD :'pw_personal';
ALTER ROLE app_web       PASSWORD :'pw_app';
ALTER ROLE app_movil     PASSWORD :'pw_app';

GRANT superadmin       TO adrian_rios;
GRANT admin            TO marco_ramirez;
GRANT support          TO carlos_rene;
GRANT seller           TO juan_pablo, erick_uriel;
GRANT not_registered   TO app_web, app_movil;
-- jose_maria: cuenta SIN rol (solo tiene INSERT directo, ver seccion 8)
```

Figura 14. Código SQL de PostgreSQL: creación de roles y cuentas personales. (continuación 2 de 2)

```
/* ============================================================
   8. PRIVILEGIOS (mismas reglas que en MySQL)
   ============================================================ */
REVOKE ALL ON DATABASE db_test FROM PUBLIC;
GRANT CONNECT ON DATABASE db_test
  TO superadmin, admin, seller, buyer, support, common, not_registered, jose_maria;

REVOKE ALL ON ALL TABLES    IN SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL FUNCTIONS IN SCHEMA public FROM PUBLIC;
GRANT USAGE ON SCHEMA public TO superadmin, admin, seller, buyer, support, common, not_registered,
    jose_maria;

-- superadmin y admin: todo sobre db_test
GRANT ALL ON ALL TABLES    IN SCHEMA public TO superadmin, admin;
GRANT ALL ON ALL SEQUENCES IN SCHEMA public TO superadmin, admin;
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO superadmin, admin;

-- support: lee y actualiza usuarios y productos, administra categorias, ejecuta funciones
GRANT SELECT, UPDATE         ON tb_users, tb_products TO support;
GRANT SELECT, INSERT, UPDATE ON tbc_categories        TO support;
GRANT SELECT ON vw_trazabilidad_logs, vw_trazabilidad_usuarios, vw_trazabilidad_productos,
    vw_trazabilidad_people TO support;
GRANT EXECUTE ON FUNCTION
    sp_register_user_with_person(VARCHAR,VARCHAR,VARCHAR,VARCHAR,VARCHAR,VARCHAR,DATE,SMALLINT,CHAR,
    VARCHAR),
                        sp_soft_delete_user(INTEGER), sp_soft_delete_product(INTEGER),
                            sp_restore_product(INTEGER) TO support;

-- seller: gestiona productos (temporal para poder importar CSV)
GRANT SELECT, INSERT, UPDATE ON tb_products TO seller;
GRANT TEMPORARY ON DATABASE db_test TO seller;

-- buyer y common: consulta del catalogo
GRANT SELECT ON tb_products, tbc_categories TO buyer, common;
```

Figura 15. Código SQL de PostgreSQL: asignación de privilegios por rol. (continuación 1 de 2)

```
-- not_registered y la cuenta sin rol: solo pueden registrarse
GRANT INSERT ON tb_users, tb_people TO not_registered, jose_maria;

SELECT 'Roles, cuentas, base de datos, tablas, triggers, vistas y privilegios creados en
    PostgreSQL' AS mensaje;
```

Figura 16. Código SQL de PostgreSQL: asignación de privilegios por rol. (continuación 2 de 2)

```
/* ============================================================
   ROLES, CUENTAS PERSONALES Y PRIVILEGIOS
   ============================================================ */
USE db_test;

/* ---- 1. Roles obligatorios ---- */
CREATE ROLE IF NOT EXISTS 'superadmin', 'admin', 'seller', 'buyer', 'support', 'common',
    'not_registered';

/* ---- 2. Privilegios de cada rol ---- */
GRANT ALL PRIVILEGES ON *.*      TO 'superadmin';
GRANT ALL PRIVILEGES ON db_test.* TO 'admin';

GRANT SELECT, UPDATE          ON db_test.tb_users      TO 'support';
GRANT SELECT, UPDATE          ON db_test.tb_products   TO 'support';
GRANT SELECT, INSERT, UPDATE  ON db_test.tbc_categories TO 'support';
GRANT EXECUTE                 ON db_test.*             TO 'support';

GRANT SELECT, INSERT, UPDATE  ON db_test.tb_products   TO 'seller';

GRANT SELECT ON db_test.tb_products    TO 'buyer', 'common';
GRANT SELECT ON db_test.tbc_categories TO 'buyer', 'common';

GRANT INSERT ON db_test.tb_users  TO 'not_registered';
GRANT INSERT ON db_test.tb_people TO 'not_registered';

/* ---- 3. Cuentas personales (5 solicitadas) y de apoyo ---- */
CREATE USER IF NOT EXISTS 'adrian.rios'@'%'    IDENTIFIED BY '__PW_PERSONAL__';
CREATE USER IF NOT EXISTS 'marco.ramirez'@'%'  IDENTIFIED BY '__PW_PERSONAL__';
CREATE USER IF NOT EXISTS 'carlos.rene'@'%'    IDENTIFIED BY '__PW_PERSONAL__';
CREATE USER IF NOT EXISTS 'juan.pablo'@'%'     IDENTIFIED BY '__PW_PERSONAL__';
CREATE USER IF NOT EXISTS 'erick.uriel'@'%'    IDENTIFIED BY '__PW_PERSONAL__';
CREATE USER IF NOT EXISTS 'jose.maria'@'%'     IDENTIFIED BY '__PW_PERSONAL__';
CREATE USER IF NOT EXISTS 'app_web'@'%'        IDENTIFIED BY '__PW_APP__';
```

Figura 17. Código SQL de MySQL: roles, cuentas personales y privilegios. (continuación 1 de 2)

```
CREATE USER IF NOT EXISTS 'app_movil'@'%'      IDENTIFIED BY '__PW_APP__';

/* ---- 4. Asignacion de roles ---- */
GRANT 'superadmin'     TO 'adrian.rios'@'%';
GRANT 'admin'          TO 'marco.ramirez'@'%';
GRANT 'support'        TO 'carlos.rene'@'%';
GRANT 'seller'         TO 'juan.pablo'@'%', 'erick.uriel'@'%';
GRANT 'not_registered' TO 'app_web'@'%', 'app_movil'@'%';

SET DEFAULT ROLE 'superadmin'     TO 'adrian.rios'@'%';
SET DEFAULT ROLE 'admin'          TO 'marco.ramirez'@'%';
SET DEFAULT ROLE 'support'        TO 'carlos.rene'@'%';
SET DEFAULT ROLE 'seller'         TO 'juan.pablo'@'%', 'erick.uriel'@'%';
SET DEFAULT ROLE 'not_registered' TO 'app_web'@'%', 'app_movil'@'%';

/* ---- 5. Cuenta SIN rol: solo puede registrarse (INSERT directo) ---- */
GRANT INSERT ON db_test.tb_users  TO 'jose.maria'@'%';
GRANT INSERT ON db_test.tb_people TO 'jose.maria'@'%';

FLUSH PRIVILEGES;

SELECT 'Base de datos, tablas, triggers, vistas, roles, cuentas y privilegios creados en MySQL' AS
    mensaje;
```

Figura 18. Código SQL de MySQL: roles, cuentas personales y privilegios. (continuación 2 de 2)

Las cuentas personales, cinco solicitadas más las de apoyo, y el rol asignado a cada una se verificaron en ambos motores.

Cuenta MySQL	Cuenta PostgreSQL	Rol
adrian.rios	adrian_rios	superadmin
marco.ramirez	marco_ramirez	admin
juan.pablo	juan_pablo	seller
erick.uriel	erick_uriel	seller
carlos.rene	carlos_rene	support
jose.maria	jose_maria	sin rol (solo INSERT en tb_users y tb_people)
app_web, app_movil	app_web, app_movil	not_registered

```
user240165@srv2060502:~$ bash scripts/07_aplicar_en_vivo.sh   # verificación en MySQL
tablas    16
vistas    18
triggers    51
productos    5007
usuarios    8
-- roles:
admin
buyer
common
not_registered
seller
superadmin
support
-- cuentas y rol:
adrian.rios    superadmin
app_movil    not_registered
app_web    not_registered
carlos.rene    support
erick.uriel    seller
jose.maria    (sin rol)
juan.pablo    seller
marco.ramirez    admin
-- privilegios de los roles:
[admin]
GRANT ALL PRIVILEGES ON `db_test`.* TO `admin`@`%`
[support]
GRANT EXECUTE ON `db_test`.* TO `support`@`%`
GRANT SELECT, UPDATE ON `db_test`.`tb_products` TO `support`@`%`
GRANT SELECT, UPDATE ON `db_test`.`tb_users` TO `support`@`%`
GRANT SELECT, INSERT, UPDATE ON `db_test`.`tbc_categories` TO `support`@`%`
[seller]
GRANT SELECT, INSERT, UPDATE ON `db_test`.`tb_products` TO `seller`@`%`
[buyer]
```

Figura 19. Verificación en MySQL: objetos, roles, cuentas con su rol y privilegios de cada rol. (continuación 1 de 2)

```
GRANT SELECT ON `db_test`.`tb_products` TO `buyer`@`%`
GRANT SELECT ON `db_test`.`tbc_categories` TO `buyer`@`%`
[common]
GRANT SELECT ON `db_test`.`tb_products` TO `common`@`%`
GRANT SELECT ON `db_test`.`tbc_categories` TO `common`@`%`
[not_registered]
GRANT INSERT ON `db_test`.`tb_people` TO `not_registered`@`%`
GRANT INSERT ON `db_test`.`tb_users` TO `not_registered`@`%`
[superadmin] 1 linea(s) con privilegios globales
```

Figura 20. Verificación en MySQL: objetos, roles, cuentas con su rol y privilegios de cada rol. (continuación 2 de 2)

```
user240165@srv2060502:~$ bash scripts/03_verificar.sh   # verificación en PostgreSQL
== PostgreSQL: tablas / triggers / vistas / funciones ==
16
58
4
4
== PostgreSQL: roles y membresias ==
adrian_rios|superadmin
app_movil|not_registered
app_web|not_registered
carlos_rene|support
erick_uriel|seller
jose_maria|(sin rol)
juan_pablo|seller
marco_ramirez|admin
admin|f
buyer|f
common|f
not_registered|f
seller|f
superadmin|f
support|f
```

Figura 21. Verificación en PostgreSQL: tablas, triggers, vistas, funciones, roles y membresías.

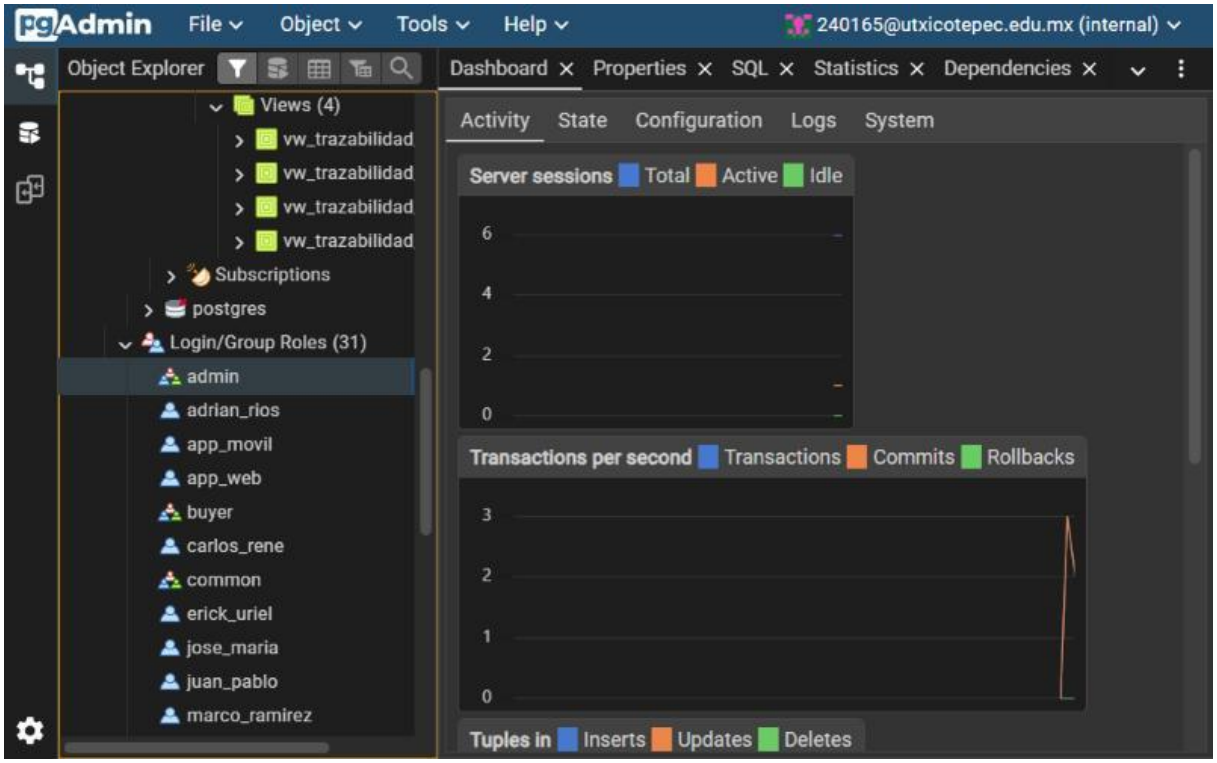


Figura 22. pgAdmin Web: Login/Group Roles de PostgreSQL con los roles y las cuentas personales.

2.7 Base de datos y objetos relacionales

En ambos motores se creó la base de datos `db_test` con 16 tablas, entre ellas `tb_users` y `tb_products`, respetando las llaves primarias, únicas y foráneas y las restricciones CHECK definidas en clase. Todas las operaciones de inserción, actualización y borrado se registran en la tabla `tb_logs` mediante triggers, con el usuario de base de datos que las ejecutó, y las vistas `vw_trazabilidad_*` filtran esa bitácora por tabla.

Objeto	MySQL 8.4	PostgreSQL 17
Tablas	16	16
Triggers	51	58 (45 de bitácora y 13 de last_update)
Vistas de trazabilidad	18	4
Procedimientos o funciones	4	4

Las claves foráneas de `tbd_cart_items` y `tbd_order_items` se definieron como llaves compuestas (proveedor y producto) en ambos motores, porque tanto PostgreSQL como MySQL 8.4 exigen que la llave referenciada sea única.

```
CREATE TABLE IF NOT EXISTS tb_users (
  id            INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  email         VARCHAR(80)  NOT NULL UNIQUE,
  nickname      VARCHAR(100) NOT NULL UNIQUE,
  password      VARCHAR(255) NOT NULL,
  creation_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  last_update   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  last_login    TIMESTAMP,
  status        BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE IF NOT EXISTS tb_people (
  person_id      BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id        INTEGER UNIQUE,
  courtesy_title VARCHAR(20),
  given_names    VARCHAR(150) NOT NULL,
  family_names   VARCHAR(150),
  birth_date     DATE,
  gender_id      SMALLINT,
  country_code   CHAR(2),
  phone          VARCHAR(25),
  status         BOOLEAN DEFAULT TRUE,
  creation_date  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  last_update    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_persons_users FOREIGN KEY (user_id) REFERENCES tb_users (id)
);


CREATE TABLE IF NOT EXISTS tb_products (
  id            INTEGER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  sku           VARCHAR(50)  NOT NULL UNIQUE,
  name          VARCHAR(250) NOT NULL,
  description   TEXT,
  current_price NUMERIC(10,2) NOT NULL DEFAULT 0.00,
```

Figura 23. Código SQL de PostgreSQL: tablas tb_users y tb_products. (continuación 1 de 2)

```
  current_stock INTEGER NOT NULL DEFAULT 0 CHECK (current_stock >= 0),
  status        BOOLEAN DEFAULT TRUE,
  creation_date TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  last_update   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);
```

Figura 24. Código SQL de PostgreSQL: tablas tb_users y tb_products. (continuación 2 de 2)

```sql
CREATE TABLE IF NOT EXISTS `tb_users` (
  `ID` int NOT NULL AUTO_INCREMENT,
  `email` varchar(80) NOT NULL,
  `nickname` varchar(100) NOT NULL,
  `password` varchar(255) NOT NULL,
  `creation_date` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `last_update` datetime DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  `last_login` datetime DEFAULT NULL,
  `status` bit(1) NOT NULL DEFAULT b'1',
  PRIMARY KEY (`ID`),
  UNIQUE KEY `email` (`email`),
  UNIQUE KEY `nickname` (`nickname`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;

CREATE TABLE IF NOT EXISTS `tb_products` (
  `id` int unsigned NOT NULL AUTO_INCREMENT,
  `sku` varchar(50) NOT NULL,
  `name` varchar(250) NOT NULL,
  `description` text,
  `current_price` decimal(10,2) NOT NULL DEFAULT '0.00',
  `current_stock` int unsigned NOT NULL DEFAULT '0',
  `status` bit(1) DEFAULT b'1',
  `creation_date` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP,
  `last_update` datetime NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  PRIMARY KEY (`id`),
  UNIQUE KEY `sku` (`sku`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

Figura 25. Código SQL de MySQL: tablas tb_users y tb_products.

```
CREATE OR REPLACE FUNCTION fn_log_operation() RETURNS trigger
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
  v_row  jsonb;
  v_op   log_operation;
  v_desc text;
BEGIN
  IF TG_OP = 'INSERT' THEN
    v_op := 'Create'; v_row := to_jsonb(NEW) - 'password';
    v_desc := format('Insercion en %s: %s', TG_TABLE_NAME, v_row::text);
  ELSIF TG_OP = 'UPDATE' THEN
    v_op := 'Update'; v_row := to_jsonb(NEW) - 'password';
    v_desc := format('Actualizacion en %s: antes=%s | despues=%s', TG_TABLE_NAME, (to_jsonb(OLD) -
        'password')::text, v_row::text);
  ELSE
    v_op := 'Delete'; v_row := to_jsonb(OLD) - 'password';
    v_desc := format('Eliminacion en %s: %s', TG_TABLE_NAME, v_row::text);
  END IF;

  INSERT INTO tb_logs (table_name, operation, db_user, description)
  VALUES (TG_TABLE_NAME, v_op,
          session_user || '@' || COALESCE(host(inet_client_addr()), 'local'),
          v_desc);
  RETURN NULL;
END $$;

DO $$
DECLARE t text;
BEGIN
  FOR t IN SELECT table_name FROM information_schema.columns
           WHERE table_schema = 'public' AND column_name = 'last_update'
  LOOP
    EXECUTE format('DROP TRIGGER IF EXISTS trg_%1$s_last_update ON %1$I', t);
    EXECUTE format('CREATE TRIGGER trg_%1$s_last_update BEFORE UPDATE ON %1$I FOR EACH ROW EXECUTE
```

Figura 26. Código SQL de PostgreSQL: función y triggers que registran las operaciones en tb_logs. (continuación 1 de 2)

```
        FUNCTION fn_set_last_update()', t);
  END LOOP;

  FOR t IN SELECT table_name FROM information_schema.tables
           WHERE table_schema = 'public' AND table_type = 'BASE TABLE' AND table_name <> 'tb_logs'
  LOOP
    EXECUTE format('DROP TRIGGER IF EXISTS trg_%1$s_log ON %1$I', t);
    EXECUTE format('CREATE TRIGGER trg_%1$s_log AFTER INSERT OR UPDATE OR DELETE ON %1$I FOR EACH
        ROW EXECUTE FUNCTION fn_log_operation()', t);
  END LOOP;
END $$;
```

Figura 27. Código SQL de PostgreSQL: función y triggers que registran las operaciones en tb_logs. (continuación 2 de 2)

```
DROP TRIGGER IF EXISTS `trg_users_after_insert`;
DELIMITER ;;
CREATE TRIGGER `trg_users_after_insert` AFTER INSERT ON `tb_users` FOR EACH ROW BEGIN
    INSERT INTO tb_logs (
        table_name,
        operation,
        db_user,
        description,
        operation_date,
        operation_status
    )
    VALUES (
        'tb_users',
        'Create',
        USER(),
        CONCAT(
            'Usuario creado. ID=', NEW.ID,
            ', email=', NEW.email,
            ', nickname=', NEW.nickname
        ),
        CURRENT_TIMESTAMP,
        b'1'
    );
END;;
```

Figura 28. Código SQL de MySQL: trigger de inserción sobre tb_users.

```
CREATE OR REPLACE VIEW vw_trazabilidad_logs AS
  SELECT id, table_name, operation, db_user, description, operation_date, operation_status FROM
      tb_logs;

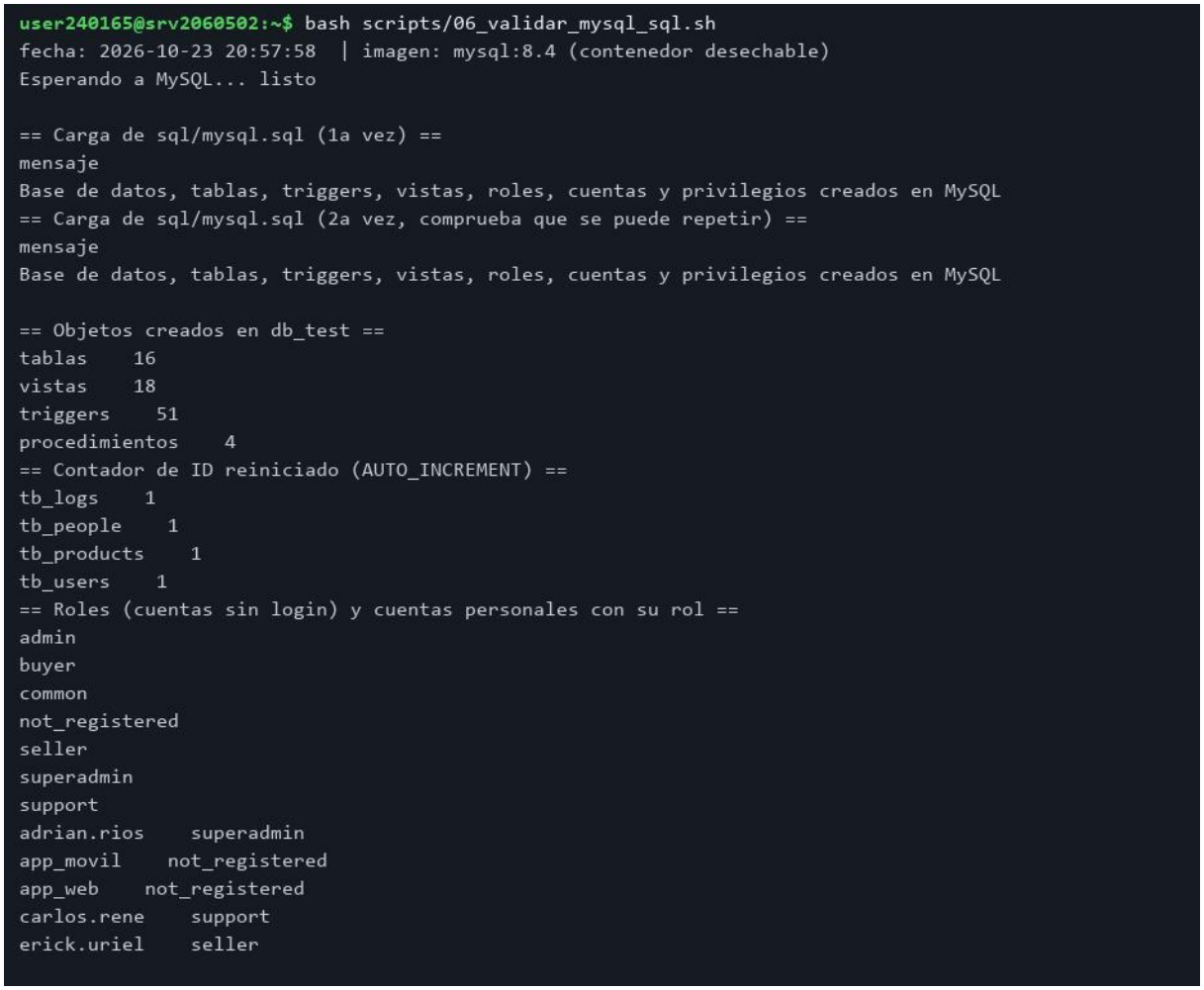
CREATE OR REPLACE VIEW vw_trazabilidad_usuarios AS
  SELECT id, table_name, operation, db_user, description, operation_date, operation_status
  FROM tb_logs WHERE table_name = 'tb_users' AND operation_status;

CREATE OR REPLACE VIEW vw_trazabilidad_productos AS
  SELECT id, table_name, operation, db_user, description, operation_date, operation_status
  FROM tb_logs WHERE table_name = 'tb_products' AND operation_status;

CREATE OR REPLACE VIEW vw_trazabilidad_people AS
  SELECT id, table_name, operation, db_user, description, operation_date, operation_status
  FROM tb_logs WHERE table_name = 'tb_people' AND operation_status;
```

Figura 29. Código SQL de PostgreSQL: vistas de trazabilidad.

La creación del esquema completo se validó cargando `sql/mysql.sql` dos veces seguidas en un contenedor MySQL desechable, sin errores. La salida muestra los objetos creados, los contadores de identificadores en 1 y el funcionamiento del trigger y la vista de trazabilidad.



```
user240165@srv2060502:~$ bash scripts/06_validar_mysql_sql.sh
fecha: 2026-10-23 20:57:58  | imagen: mysql:8.4 (contenedor desechable)
Esperando a MySQL... listo

== Carga de sql/mysql.sql (1a vez) ==
mensaje
Base de datos, tablas, triggers, vistas, roles, cuentas y privilegios creados en MySQL
== Carga de sql/mysql.sql (2a vez, comprueba que se puede repetir) ==
mensaje
Base de datos, tablas, triggers, vistas, roles, cuentas y privilegios creados en MySQL

== Objetos creados en db_test ==
tablas    16
vistas    18
triggers    51
procedimientos    4
== Contador de ID reiniciado (AUTO_INCREMENT) ==
tb_logs    1
tb_people    1
tb_products    1
tb_users    1
== Roles (cuentas sin login) y cuentas personales con su rol ==
admin
buyer
common
not_registered
seller
superadmin
support
adrian.rios    superadmin
app_movil    not_registered
app_web    not_registered
carlos.rene    support
erick.uriel    seller
```

Figura 30. Validación de sql/mysql.sql en un contenedor MySQL desechable. (continuación 1 de 2)



```
jose.maria    (sin rol)
juan.pablo    seller
marco.ramirez    admin
```

Figura 31. Validación de sql/mysql.sql en un contenedor MySQL desechable. (continuación 2 de 2)

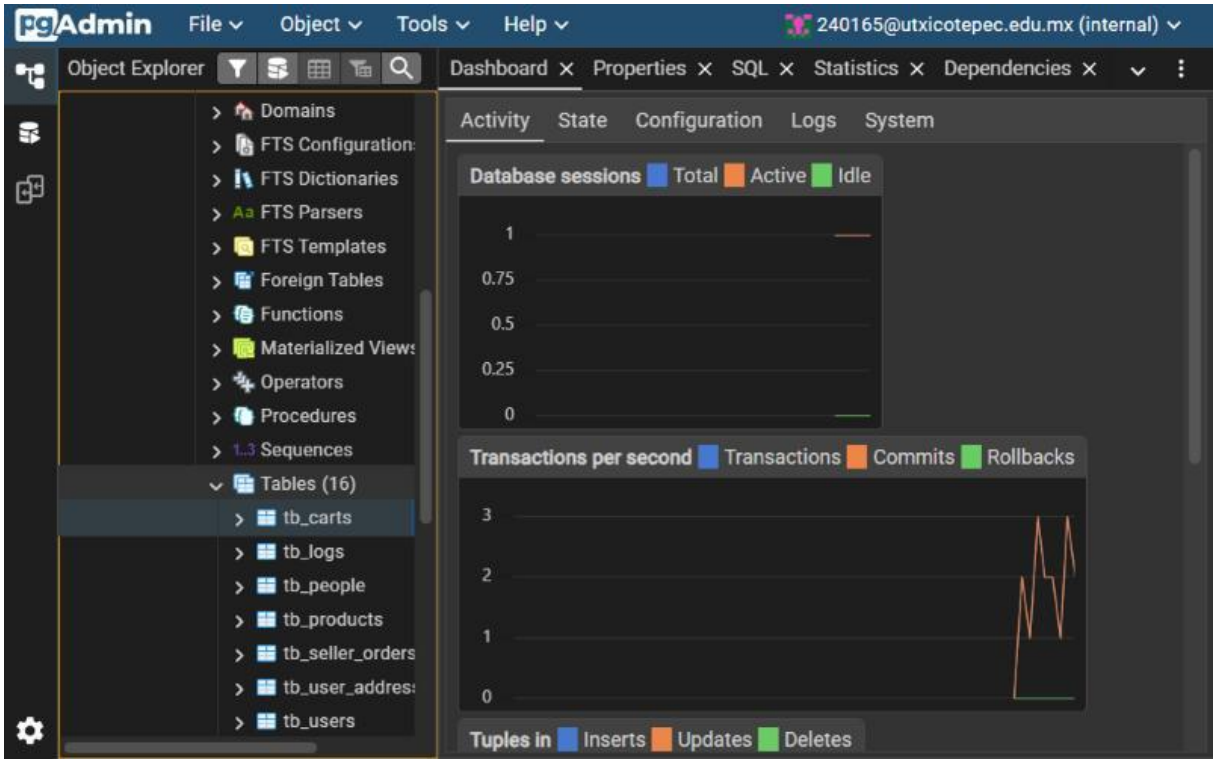


Figura 32. pgAdmin Web: Tables (16) de db_test.

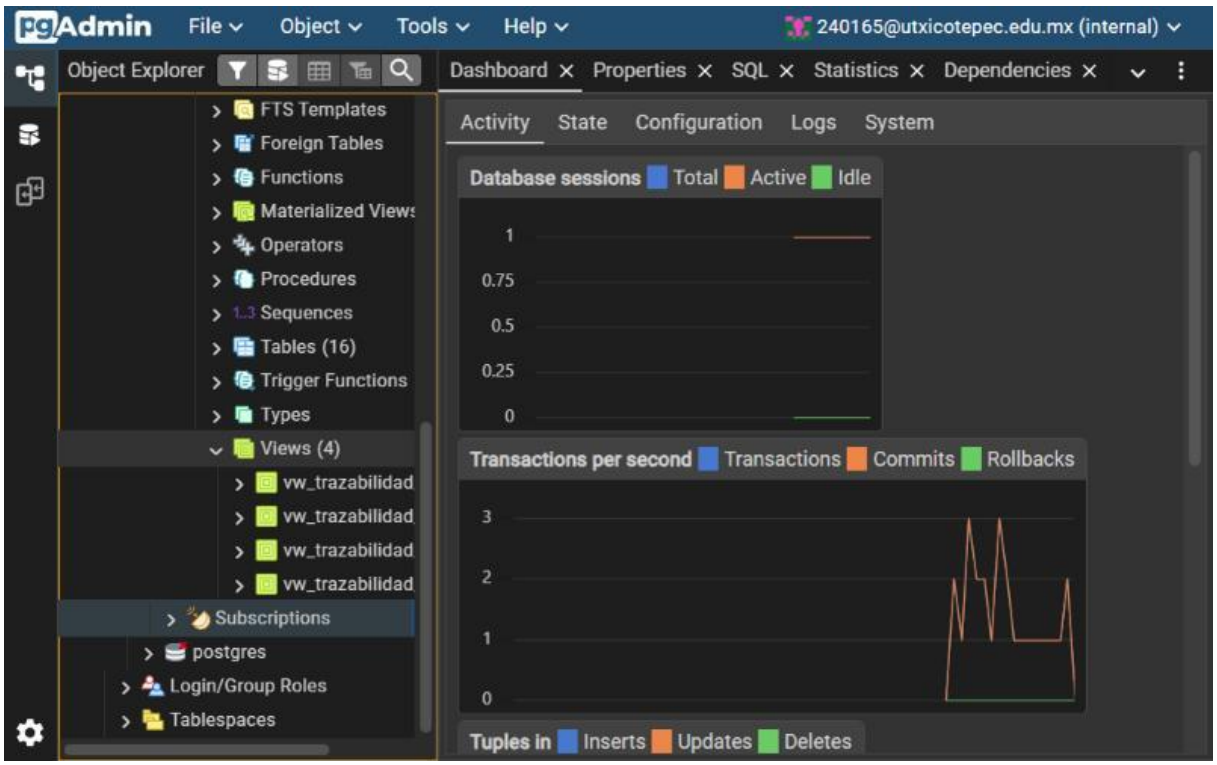


Figura 33. pgAdmin Web: esquema public de db_test con Tables (16), Trigger Functions y Views (4).

3. Pruebas y resultados

Los nueve ejercicios proporcionados en clase para `tb_users` y `tb_products` se ejecutaron con la cuenta real de cada rol, tanto en MySQL como en PostgreSQL. Los errores de permiso marcados como esperados son la prueba de que el privilegio se niega. La tabla resume cada caso y, a continuación, se muestra la operación SQL y la salida obtenida en cada motor.

Caso	Cuenta (rol)	Operación	Resultado
01	adrian (superadmin)	Insertar 3 usuarios	Correcto en ambos motores
02	jose.maria (sin rol)	Insertar 3 usuarios	Correcto en ambos motores
03	carlos.rene (support)	Insertar 3 usuarios con la rutina de registro	Correcto en ambos motores
04	adrian (superadmin)	Modificar y eliminar un usuario ajeno	Correcto en ambos motores
05	jose.maria (sin rol)	Modificar y eliminar	Denegado, como se esperaba
06	carlos.rene (support)	Modificar y eliminar	UPDATE correcto y DELETE denegado
07	adrian (superadmin)	Insertar 2 productos	Correcto en ambos motores
08	juan.pablo (seller)	Insertar 5 productos	Correcto en ambos motores
09	erick.uriel (seller)	Importar 5000 productos del CSV	Correcto: 5007 productos

Estado final de cada motor tras los nueve casos: 8 usuarios, 3 personas y 5007 productos (2 del superadmin, 5 del vendedor 1 y 5000 del archivo CSV).

Caso 01: el superadmin inserta 3 usuarios

```
-- CASO 01: superadmin (adrian_rios) inserta 3 usuarios

[adrian_rios]> INSERT INTO tb_users (email,nickname,password) VALUES
                ('root_u1@test.com','root_u1',md5('Pass123*'));
INSERT 0 1

[adrian_rios]> INSERT INTO tb_users (email,nickname,password) VALUES
                ('root_u2@test.com','root_u2',md5('Pass123*'));
INSERT 0 1

[adrian_rios]> INSERT INTO tb_users (email,nickname,password) VALUES
                ('root_u3@test.com','root_u3',md5('Pass123*'));
INSERT 0 1
```

Figura 34. Caso 01: el superadmin inserta 3 usuarios: ejecución en PostgreSQL.

```
[adrian.rios]> INSERT INTO tb_users (email,nickname,password) VALUES
                  ('root_u1@test.com','root_u1',MD5('Pass123*'));

[adrian.rios]> INSERT INTO tb_users (email,nickname,password) VALUES
                  ('root_u2@test.com','root_u2',MD5('Pass123*'));

[adrian.rios]> INSERT INTO tb_users (email,nickname,password) VALUES
                  ('root_u3@test.com','root_u3',MD5('Pass123*'));
```

Figura 35. Caso 01: el superadmin inserta 3 usuarios: ejecución en MySQL.

Caso 02: el usuario sin rol inserta 3 usuarios

```
[jose_maria]> INSERT INTO tb_users (email,nickname,password) VALUES
                  ('norol_u1@test.com','norol_u1',md5('Pass123*'));
INSERT 0 1

[jose_maria]> INSERT INTO tb_users (email,nickname,password) VALUES
                  ('norol_u2@test.com','norol_u2',md5('Pass123*'));
INSERT 0 1

[jose_maria]> INSERT INTO tb_users (email,nickname,password) VALUES
                  ('norol_u3@test.com','norol_u3',md5('Pass123*'));
INSERT 0 1
```

Figura 36. Caso 02: el usuario sin rol inserta 3 usuarios: ejecución en PostgreSQL.

```
[jose.maria]> INSERT INTO tb_users (email,nickname,password) VALUES
                  ('norol_u1@test.com','norol_u1',MD5('Pass123*'));

[jose.maria]> INSERT INTO tb_users (email,nickname,password) VALUES
                  ('norol_u2@test.com','norol_u2',MD5('Pass123*'));

[jose.maria]> INSERT INTO tb_users (email,nickname,password) VALUES
                  ('norol_u3@test.com','norol_u3',MD5('Pass123*'));
```

Figura 37. Caso 02: el usuario sin rol inserta 3 usuarios: ejecución en MySQL.

Caso 03: support inserta 3 usuarios

```
[carlos_rene]> SELECT sp_register_user_with_person('support_u1@test.com','support_u1','Pass123*','Sr
                ','Support1','Prueba','2000-01-01',1::smallint,'MX','5550000001');
 sp_register_user_with_person
------------------------------
                            7
(1 row)


[carlos_rene]> SELECT sp_register_user_with_person('support_u2@test.com','support_u2','Pass123*','Sr
                ','Support2','Prueba','2000-01-02',1::smallint,'MX','5550000002');
 sp_register_user_with_person
------------------------------
                            8
(1 row)


[carlos_rene]> SELECT sp_register_user_with_person('support_u3@test.com','support_u3','Pass123*','Sr
                ','Support3','Prueba','2000-01-03',1::smallint,'MX','5550000003');
 sp_register_user_with_person
------------------------------
                            9
(1 row)
```

Figura 38. Caso 03: support inserta 3 usuarios: ejecución en PostgreSQL.

```
[carlos.rene]> CALL sp_register_user_with_person('support_u1@test.com','support_u1','Pass123*','Sr',
                'Support1','Prueba','2000-01-01',1,'MX','5550000001');

[carlos.rene]> CALL sp_register_user_with_person('support_u2@test.com','support_u2','Pass123*','Sr',
                'Support2','Prueba','2000-01-02',1,'MX','5550000002');

[carlos.rene]> CALL sp_register_user_with_person('support_u3@test.com','support_u3','Pass123*','Sr',
                'Support3','Prueba','2000-01-03',1,'MX','5550000003');
```

Figura 39. Caso 03: support inserta 3 usuarios: ejecución en MySQL.

Caso 04: el superadmin modifica y elimina un usuario ajeno

```
[adrian_rios]> UPDATE tb_users SET nickname='norol_u1_mod' WHERE email='norol_u1@test.com';
UPDATE 1

[adrian_rios]> DELETE FROM tb_users WHERE email='norol_u3@test.com';
DELETE 1
```

Figura 40. Caso 04: el superadmin modifica y elimina un usuario ajeno: ejecución en PostgreSQL.

```
[adrian.rios]> UPDATE tb_users SET nickname='norol_u1_mod' WHERE email='norol_u1@test.com';

[adrian.rios]> DELETE FROM tb_users WHERE email='norol_u3@test.com';
```

Figura 41. Caso 04: el superadmin modifica y elimina un usuario ajeno: ejecución en MySQL.

Caso 05: el usuario sin rol intenta modificar y eliminar

Los dos errores de permiso son el resultado esperado: la cuenta sin rol solo tiene el privilegio INSERT.

```
[jose_maria]> UPDATE tb_users SET nickname='hack' WHERE email='root_u1@test.com';
ERROR:  permission denied for table tb_users

[jose_maria]> DELETE FROM tb_users WHERE email='root_u1@test.com';
ERROR:  permission denied for table tb_users
```

Figura 42. Caso 05: el usuario sin rol intenta modificar y eliminar: ejecución en PostgreSQL.

```
[jose.maria]> UPDATE tb_users SET nickname='hack' WHERE email='root_u1@test.com';
ERROR 1142 (42000) at line 1: UPDATE command denied to user 'jose.maria'@'127.0.0.1' for table
    'tb_users'

[jose.maria]> DELETE FROM tb_users WHERE email='root_u1@test.com';
ERROR 1142 (42000) at line 1: DELETE command denied to user 'jose.maria'@'127.0.0.1' for table
    'tb_users'
```

Figura 43. Caso 05: el usuario sin rol intenta modificar y eliminar: ejecución en MySQL.

Caso 06: support modifica y elimina

La actualización se permite, pero la eliminación se niega: support solo tiene SELECT y UPDATE.

```
[carlos_rene]> UPDATE tb_users SET nickname='root_u2_mod_support' WHERE email='root_u2@test.com';
UPDATE 1

[carlos_rene]> DELETE FROM tb_users WHERE email='root_u2@test.com';
ERROR:  permission denied for table tb_users
```

Figura 44. Caso 06: support modifica y elimina: ejecución en PostgreSQL.

```
[carlos.rene]> UPDATE tb_users SET nickname='root_u2_mod_support' WHERE email='root_u2@test.com';

[carlos.rene]> DELETE FROM tb_users WHERE email='root_u2@test.com';
ERROR 1142 (42000) at line 1: DELETE command denied to user 'carlos.rene'@'127.0.0.1' for table
    'tb_users'
```

Figura 45. Caso 06: support modifica y elimina: ejecución en MySQL.

Caso 07: el superadmin inserta 2 productos

```
[adrian_rios]> INSERT INTO tb_products (sku,name,description,current_price,current_stock) VALUES
                 ('ROOT-0001','Producto root 1','Insertado por
                 root',100.00,10),('ROOT-0002','Producto root 2','Insertado por root',200.00,20);
INSERT 0 2
```

Figura 46. Caso 07: el superadmin inserta 2 productos: ejecución en PostgreSQL.

```
[adrian.rios]> INSERT INTO tb_products (sku,name,description,current_price,current_stock) VALUES
                 ('ROOT-0001','Producto root 1','Insertado por
                 root',100.00,10),('ROOT-0002','Producto root 2','Insertado por root',200.00,20);
```

Figura 47. Caso 07: el superadmin inserta 2 productos: ejecución en MySQL.

Caso 08: el vendedor 1 inserta 5 productos

```
[juan_pablo]> INSERT INTO tb_products (sku,name,description,current_price,current_stock) VALUES
                ('V1-0001','Producto V1-1','Vendedor 1',10.00,5),('V1-0002','Producto
                V1-2','Vendedor 1',20.00,5),('V1-0003','Producto V1-3','Vendedor
                1',30.00,5),('V1-0004','Producto V1-4','Vendedor 1',40.00,5),('V1-0005','Producto
                V1-5','Vendedor 1',50.00,5);
INSERT 0 5
```

Figura 48. Caso 08: el vendedor 1 inserta 5 productos: ejecución en PostgreSQL.

```
[juan.pablo]> INSERT INTO tb_products (sku,name,description,current_price,current_stock) VALUES
                ('V1-0001','Producto V1-1','Vendedor 1',10.00,5),('V1-0002','Producto
                V1-2','Vendedor 1',20.00,5),('V1-0003','Producto V1-3','Vendedor
                1',30.00,5),('V1-0004','Producto V1-4','Vendedor 1',40.00,5),('V1-0005','Producto
                V1-5','Vendedor 1',50.00,5);
```

Figura 49. Caso 08: el vendedor 1 inserta 5 productos: ejecución en MySQL.

Caso 09: el vendedor 2 importa el CSV de 5000 productos

El primer intento falló porque 1666 filas del CSV traen `last_update` vacío y la columna es obligatoria en PostgreSQL; se corrigió usando la fecha de creación cuando falta (el intento fallido se muestra al final de esta sección).

```
(Si last_update viene vacio en el CSV se usa creation_date: la columna es NOT NULL)
[erick_uriel]> CREATE TEMP TABLE + \copy del CSV + INSERT ... SELECT
CREATE TABLE
COPY 5000
INSERT 0 5000
```

Figura 50. Caso 09: el vendedor 2 importa el CSV de 5000 productos: ejecución en PostgreSQL.

```
[erick.uriel]> LOAD DATA LOCAL INFILE '/tmp/tb_products_5000.csv' INTO TABLE tb_products CHARACTER
                  SET utf8mb4 FIELDS TERMINATED BY ',' OPTIONALLY ENCLOSED BY '"' LINES TERMINATED
                  BY '\r\n' IGNORE 1 LINES
                  (@id,sku,name,description,current_price,current_stock,@st,creation_date,last_upda
                  te) SET status=@st+0;

-- RESUMEN (root)

[adrian.rios]> SELECT COUNT(*) AS usuarios FROM tb_users; SELECT COUNT(*) AS personas FROM
                  tb_people; SELECT COUNT(*) AS productos FROM tb_products; SELECT
                  id,email,nickname FROM tb_users;
usuarios
8
personas
3
productos
5007
id    email    nickname
1    root_u1@test.com    root_u1
2    root_u2@test.com    root_u2_mod_support
3    root_u3@test.com    root_u3
4    norol_u1@test.com    norol_u1_mod
5    norol_u2@test.com    norol_u2
7    support_u1@test.com    support_u1
8    support_u2@test.com    support_u2
9    support_u3@test.com    support_u3

[adrian.rios]> SELECT db_user, operation, table_name, COUNT(*) total FROM tb_logs GROUP BY db_user,
                  operation, table_name ORDER BY db_user;
db_user    operation    table_name    total
adrian.rios@127.0.0.1    Create    tb_products    4
adrian.rios@127.0.0.1    Create    tb_users    6
```

Figura 51. Caso 09: el vendedor 2 importa el CSV de 5000 productos: ejecución en MySQL. (continuación 1 de 2)

```
adrian.rios@127.0.0.1    Update    tb_users    2
adrian.rios@127.0.0.1    Delete    tb_users    2
carlos.rene@127.0.0.1    Create    tb_people    3
carlos.rene@127.0.0.1    Create    tb_users    6
carlos.rene@127.0.0.1    Update    tb_users    2
erick.uriel@127.0.0.1    Create    tb_products    10000
jose.maria@127.0.0.1    Create    tb_users    6
juan.pablo@127.0.0.1    Create    tb_products    10
```

Figura 52. Caso 09: el vendedor 2 importa el CSV de 5000 productos: ejecución en MySQL. (continuación 2 de 2)

Resumen y trazabilidad tras los casos

La bitácora `tb_logs` registra cada operación con el usuario que la realizó, y la vista `vw_trazabilidad_productos` muestra los 5007 productos insertados.

```
[adrian_rios]> SELECT (SELECT count(*) FROM tb_users) AS usuarios, (SELECT count(*) FROM tb_people)
                  AS personas, (SELECT count(*) FROM tb_products) AS productos;
 usuarios | personas | productos
----------+----------+-----------
        8 |        3 |      5007
(1 row)


[adrian_rios]> SELECT db_user, operation, table_name, count(*) FROM tb_logs GROUP BY 1,2,3 ORDER BY
                  1,3,2;
        db_user         | operation | table_name  | count
------------------------+-----------+-------------+-------
 adrian_rios@172.16.1.3 | Create    | tb_products |     2
 adrian_rios@172.16.1.3 | Create    | tb_users    |     3
 adrian_rios@172.16.1.3 | Update    | tb_users    |     1
 adrian_rios@172.16.1.3 | Delete    | tb_users    |     1
 carlos_rene@172.16.1.3 | Create    | tb_people   |     3
 carlos_rene@172.16.1.3 | Create    | tb_users    |     3
 carlos_rene@172.16.1.3 | Update    | tb_users    |     1
 erick_uriel@172.16.1.3 | Create    | tb_products |  5000
 jose_maria@172.16.1.3  | Create    | tb_users    |     3
 juan_pablo@172.16.1.3  | Create    | tb_products |     5
(10 rows)


[adrian_rios]> SELECT count(*) AS productos_en_vista_trazabilidad FROM vw_trazabilidad_productos;
 productos_en_vista_trazabilidad
---------------------------------
                            5007
(1 row)
```

Figura 53. Resumen y trazabilidad en PostgreSQL tras el caso 09.

Primer intento fallido del caso 09

```
[erick_uriel]> CREATE TEMP TABLE + \copy del CSV + INSERT ... SELECT
CREATE TABLE
COPY 5000
ERROR:  null value in column "last_update" of relation "tb_products" violates not-null constraint
DETAIL:  Failing row contains (10, PROD-MON-00003, Audífonos Sony WH-1000XM5 Cancelación de Ruido
    Lite - Azul Mar..., Audífonos inalámbricos Over-Ear con la mejor cancelación de r..., 7348.95,
    39, t, 2025-01-22 03:03:00, null).
```

Figura 54. Primer intento del caso 09 en PostgreSQL, conservado como evidencia.

4. Control de versiones

El trabajo se entrega en un repositorio **privado** de GitHub (`Manual_VPS_BDs_Relacionales_240165`, cuenta AdrianRGZ123). Se creó la rama `Manual_VPS_BDs_Relacionales` a partir de `main`, se registraron los cambios en commits separados por contenido y la rama se fusionó con `main` sin conflictos pendientes. El archivo `.env` con las contraseñas está excluido del repositorio mediante `.gitignore`.

```
PS> git init -b main
PS> git add .gitignore
PS> git commit -m "Inicio del repositorio"
PS> git checkout -b Manual_VPS_BDs_Relacionales
PS> git add docker-compose.yml .env.example
PS> git commit -m "Docker Compose con PostgreSQL, MySQL y pgAdmin"
PS> git add sql/postgresql.sql
PS> git commit -m "Script SQL de PostgreSQL"
PS> git add sql/mysql.sql
PS> git commit -m "Script SQL de MySQL"
PS> git add scripts seeds
PS> git commit -m "Scripts de despliegue, carga y casos de prueba"
PS> git add evidencias
PS> git commit -m "Evidencias de ejecucion"
PS> git add README.md
PS> git commit -m "README del entregable"
PS> git checkout main
PS> git merge --no-ff Manual_VPS_BDs_Relacionales
PS> git remote add origin https://github.com/AdrianRGZ123/Manual_VPS_BDs_Relacionales_240165.git
PS> git push -u origin main Manual_VPS_BDs_Relacionales
```

Figura 55. Comandos de Git: creación de la rama, commits, fusión y envío al repositorio.

```
PS> git checkout main
Switched to branch 'main'
PS> git merge --no-ff Manual_VPS_BDs_Relacionales
Merge made by the 'ort' strategy.
 .env.example                                |    9 +
 .gitignore                                  |    1 -
 README.md                                   |   73 +
 docker-compose.yml                          |   55 +
 evidencias/aplicar_en_vivo.log              |  103 +
 evidencias/conexion_remota_externa.log      |   24 +
 evidencias/mysql_validacion_mysql_sql.log   |   74 +
 evidencias/postgresql_caso09_importar_csv.log |   79 +
 evidencias/postgresql_casos_prueba.log      |  143 +
 evidencias/vps_info.log                     |   49 +
 scripts/01_deploy_compose.sh                |   10 +
 scripts/02_cargar_postgres.sh               |    9 +
 scripts/03_verificar.sh                     |   18 +
 scripts/04_casos_prueba_postgres.sh         |   52 +
 scripts/05_caso09_importar_csv_postgres.sh  |   26 +
 scripts/06_validar_mysql_sql.sh             |   42 +
 scripts/07_aplicar_en_vivo.sh               |   63 +
 scripts/08_info_vps.sh                      |   20 +
 seeds/categorias_ecommerce_100.csv          |  101 +
 seeds/tb_products_5000.csv                  | 5001 ++++++++++++++++++++++++
 sql/mysql.sql                               | 2590 ++++++++++++++
 sql/postgresql.sql                          |  449 +++
 22 files changed, 8990 insertions(+), 1 deletion(-)
```

Figura 56. Fusión de la rama de trabajo con main sin conflictos.

```
PS> git branch -a
  Manual_VPS_BDs_Relacionales
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/Manual_VPS_BDs_Relacionales
  remotes/origin/main
PS> git remote -v
origin    https://github.com/AdrianRGZ123/Manual_VPS_BDs_Relacionales_240165.git (fetch)
origin    https://github.com/AdrianRGZ123/Manual_VPS_BDs_Relacionales_240165.git (push)
PS> git log --graph --oneline --all
*   6e4187b Fusion del proveedor del VPS en main
|\
| * f552b42 Proveedor del VPS en la portada y el README
* | aa33bb8 Fusion de la conexion remota a MySQL en main
|\|
| * d2a8a35 Captura de la conexion remota a MySQL
* | c1e4af9 Fusion de las capturas de pgAdmin en main
|\|
| * d15fd93 Capturas de pgAdmin Web y reporte actualizado
* | a7c6175 Fusion de gitattributes en main
|\|
| * 0650543 Marcar PDF e imagenes como binarios
* | 1f0c2be Fusion de grupo y equipo en main
|\|
| * ca454b7 Grupo y equipo en la portada y el README
* | 9c449e1 Fusion de la evidencia de MySQL en main
|\|
| * 2fd607d Evidencia de los casos de prueba en MySQL
* | 8c8b8a9 Fusion del reporte PDF en main
|\|
| * ea9e998 Reporte tecnico PDF y capturas
* | ebd033d Fusion de Manual_VPS_BDs_Relacionales en main
|\|
| * ecb3452 README del entregable
```

Figura 57. Ramas, repositorio remoto e historial de commits con las fusiones. (continuación 1 de 2)

```
| * 5939007 Evidencias de ejecucion
| * 21ae45d Scripts de despliegue, carga y casos de prueba
| * 87ab2aa Script SQL de MySQL
| * 4d371fc Script SQL de PostgreSQL
| * 7c52a73 Docker Compose con PostgreSQL, MySQL y pgAdmin
|/
```

Figura 58. Ramas, repositorio remoto e historial de commits con las fusiones. (continuación 2 de 2)

Archivos entregados con la estructura obligatoria del repositorio:

```
Manual_VPS_BDs_Relacionales/
    Reporte_Unidad1_240165.docx
    Reporte_Unidad1_240165.pdf
    docker-compose.yml
    sql/
        postgresql.sql
        mysql.sql
    README.md
    .env.example
    scripts/     (despliegue, carga, casos de prueba y verificación)
    evidencias/  (logs de cada ejecución)
    seeds/       (CSV de categorías y de 5000 productos)
    capturas/    (capturas de pantalla del reporte)
```

Figura 59. Estructura de ar

chivos entregados.

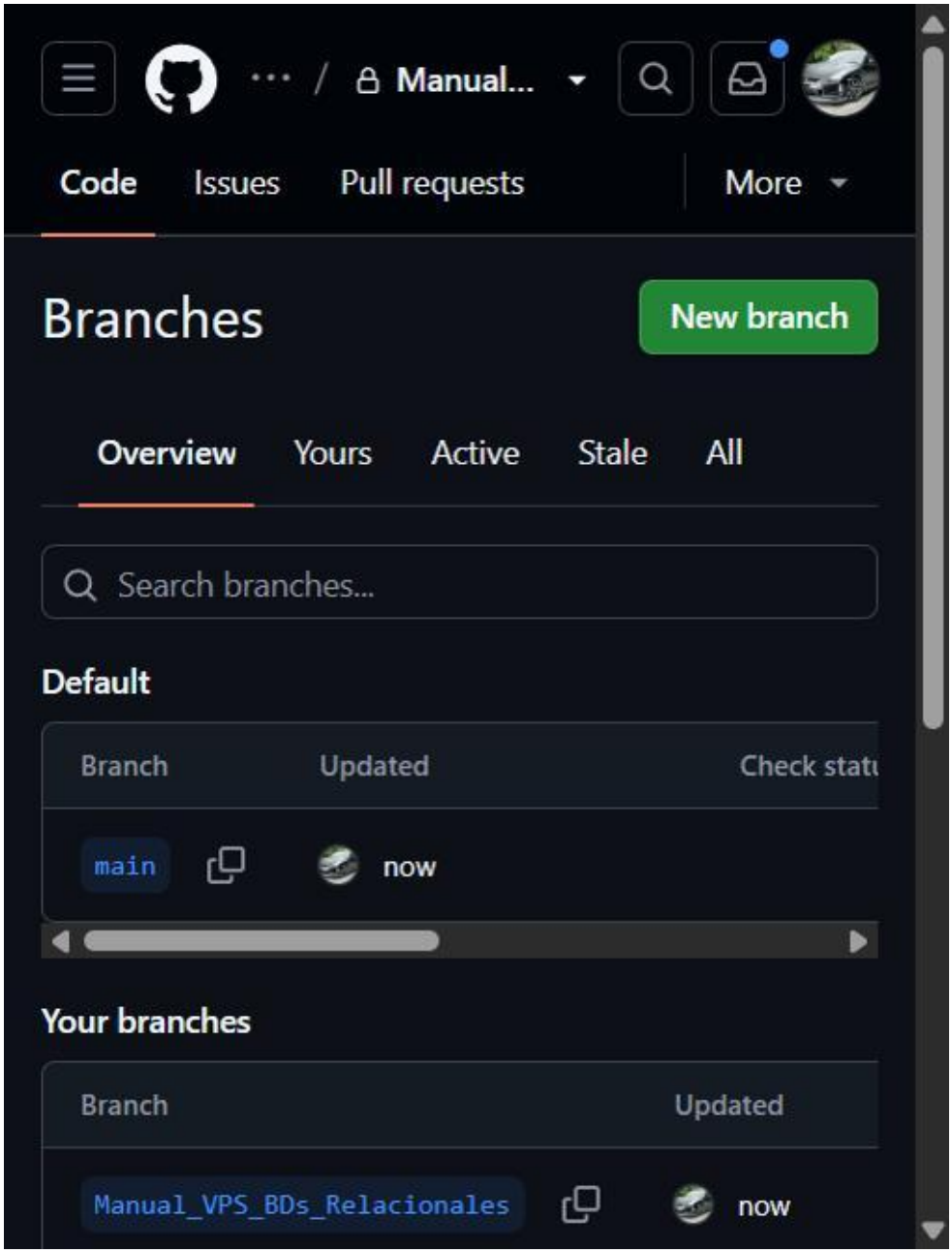


Figura 60. Repositorio privado en GitHub con la rama Manual_VPS_BDs_Relacionales.

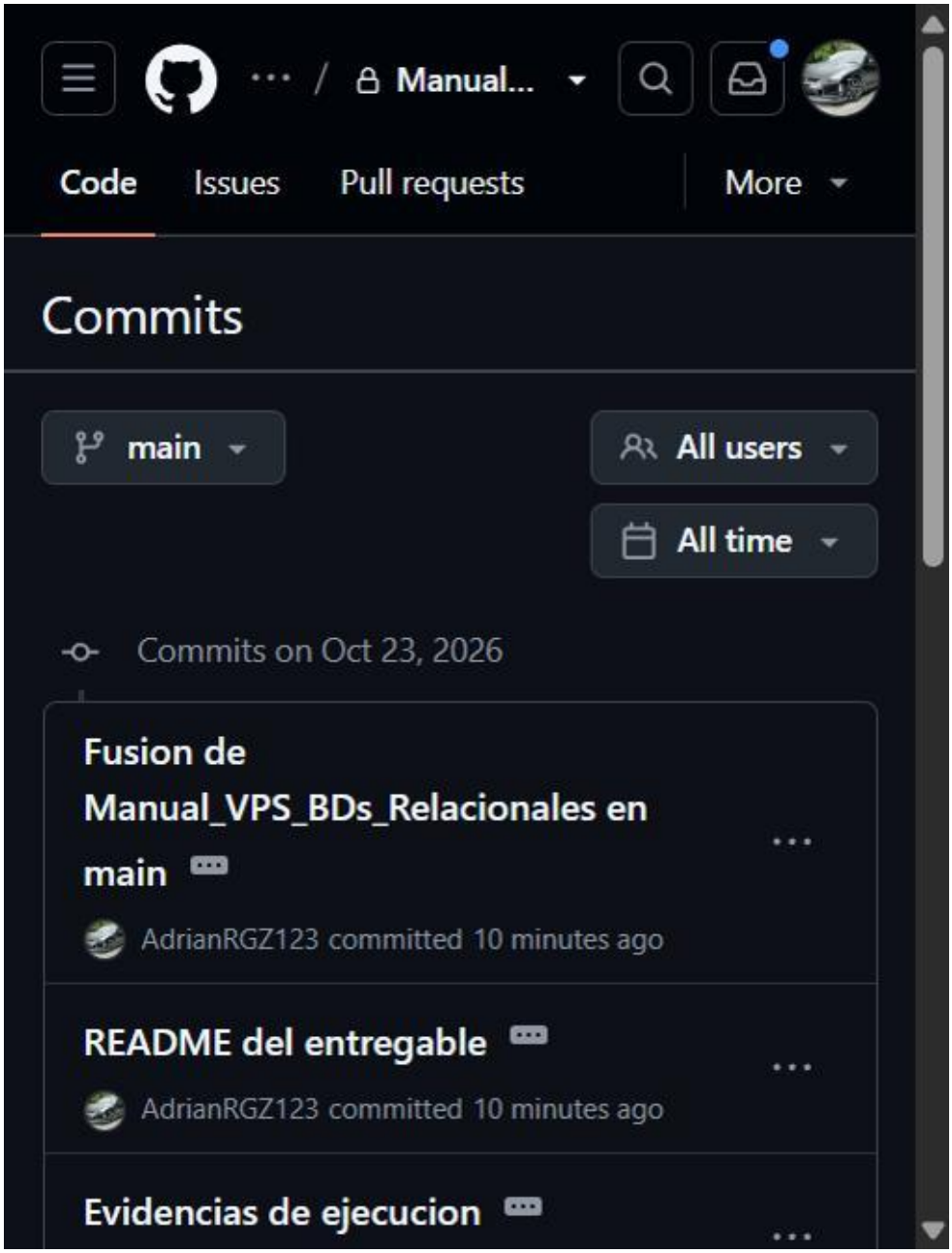


Figura 61. Historial de commits y fusión en GitHub.

5. Conclusiones personales

Competencias desarrolladas. Aprendí a desplegar una infraestructura de bases de datos completa con Docker Compose, a publicar los servicios en puertos propios, a administrar roles, cuentas y privilegios en PostgreSQL y en MySQL, y a verificar cada regla de negocio con la cuenta real de cada rol, comprobando tanto lo que se permite como lo que se niega. También practiqué el uso de triggers y vistas para dar trazabilidad y el flujo de trabajo con ramas, commits y fusión en Git.

Dificultades técnicas y soluciones. La cuenta del estudiante no tiene `sudo`, así que no se pudo editar la configuración del sistema; la solución fue ejecutar los motores en contenedores Docker y definir el puerto en el comando de cada servicio. El respaldo original venía de MySQL 8.0 y MySQL 8.4 rechazó dos llaves foráneas que apuntaban a columnas no únicas, por lo que las corregí como llaves compuestas en ambos motores. PostgreSQL y MySQL difieren en tipos de datos, autoincrementos y triggers, así que traduje el esquema y sustituí `ON UPDATE CURRENT_TIMESTAMP` por triggers. La importación de 5000 productos falló en PostgreSQL por valores vacíos en `last_update`, y la resolví completándolos con la fecha de creación. Además, detecté que algunas contraseñas habían quedado visibles en un repositorio, así que las roté y dejé las contraseñas únicamente en un archivo `.env` excluido de Git.

Reflexión. Trabajar con bases de datos en la nube obliga a pensar en la seguridad desde el inicio: puertos expuestos, contraseñas fuera del repositorio y privilegios mínimos por rol. Automatizar el despliegue y la verificación con scripts permitió repetir las pruebas sin errores y dejar evidencia reproducible de cada resultado.

Nota de seguridad. Este reporte y el repositorio no contienen contraseñas; las credenciales de prueba se entregan al docente por un canal autorizado.